

ON THIS PAGE

01 Architecture & data flow

02 Request lifecycle

03 Design tradeoffs

04 Technical insights

05 Risk management

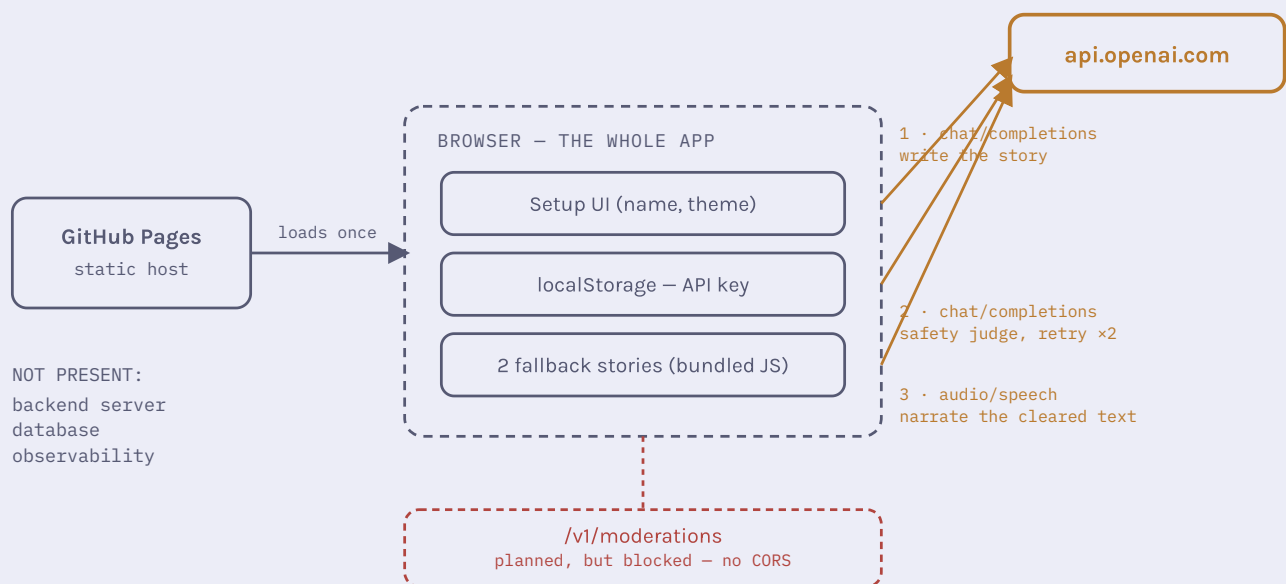
AS-BUILT · MVP

What actually shipped

One static HTML page, no backend, no database — the browser talks to OpenAI directly. Simpler than the planned architecture in every way except one: it moved every risk the backend was going to absorb onto the browser instead.

01 Architecture & data flow

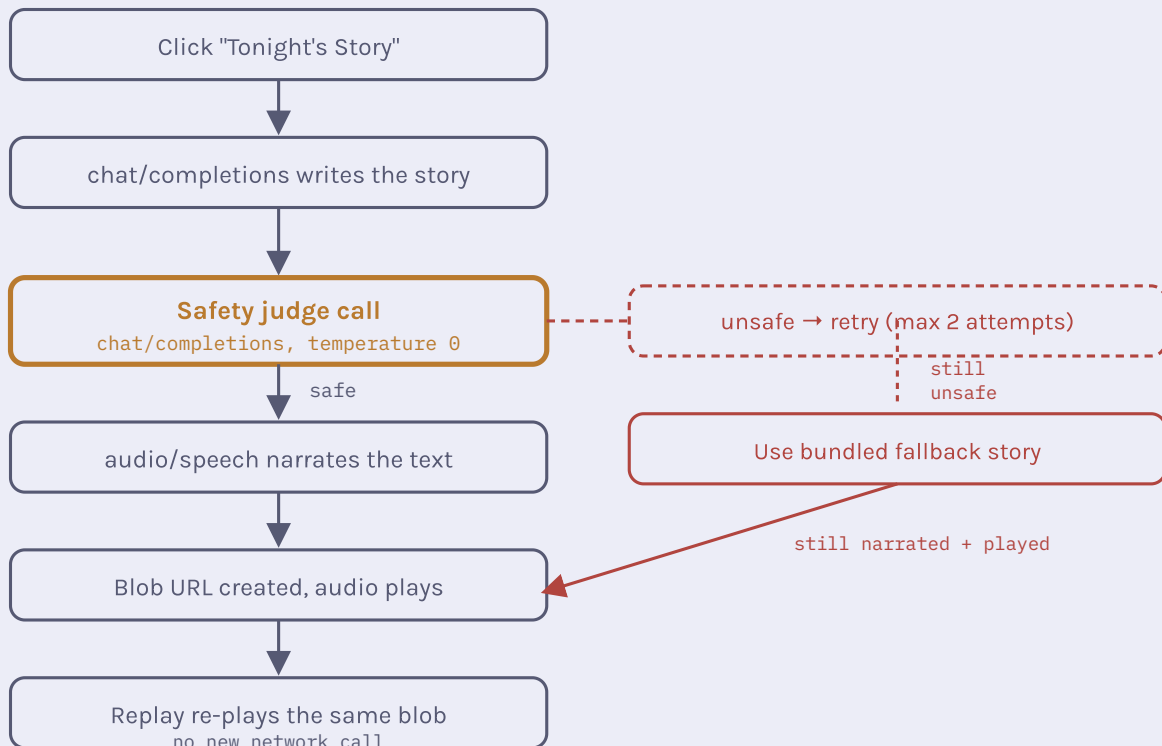
Two components, no server in between. GitHub Pages hands the browser a static file once; after that, every request goes straight from the browser to OpenAI.



Everything inside the dashed boundary runs in the visitor's own browser. The planned dedicated moderation endpoint (bottom, red) turned out to be unreachable — see §4.

02 Request lifecycle

One "Tonight's Story" click, three sequential API calls, one possible fork.



The safety judge (accent) is the one decision point in the flow — a fail there never surfaces an error to the parent, it silently swaps in a pre-written story instead.

03 Design tradeoffs

Every simplification below was made to satisfy one constraint: ship a working, deployable demo with no server to operate.

No backend at all

GAINED Zero hosting cost, zero ops, deploys as a single static file, works the moment GitHub Pages serves it.

COST No place to hide a shared API key, enforce quotas, log generations, or persist anything beyond one browser's local storage.

API key entered by the visitor, stored in localStorage

GAINED Never committed to the public repo; each visitor uses their own billing, not ours.

COST Sits in plaintext in devtools on that device. Fine for a single-user personal tool; wrong model for anything multi-tenant.

LLM-as-judge instead of the dedicated moderation endpoint

GAINED A real, working safety gate that actually runs in a browser — the planned endpoint doesn't support CORS at all.

COST A general-purpose model prompted to judge is less rigorously calibrated than OpenAI's purpose-trained classifier, and it's a second paid call instead of a typically-free one.

No streaming — full story, then full audio

GAINED Far simpler client code; nothing partially plays before the safety gate has cleared it.

COST Slower perceived start than the "time to first audio" target in the original architecture plan — this MVP waits for everything before playing anything.

No persistence beyond the API key

GAINED Nothing to design, migrate, or secure — nothing personal about the child is ever stored anywhere.

COST The PRD's "evolves over time" — recurring characters, rating-driven personalization — doesn't exist yet. Every story starts from zero.

01 Not every OpenAI endpoint is browser-reachable

chat/completions and audio/speech both send permissive CORS headers and work fine from a web page. /v1/moderations sends none – the browser blocks the preflight outright, and the failure looks exactly like a dead network connection, not a rejected request. This is the single fact that determined the safety architecture.

02 A CORS failure and a network failure are indistinguishable to JS

Both surface as a generic `TypeError: Failed to fetch`. The only way to tell them apart is the browser console, which the user obviously never sees – the app's own "couldn't reach OpenAI" message was accurate-sounding but wrong. Diagnosing this needed the actual console output, not just the error text.

03 GitHub Pages' "built" status isn't "live everywhere"

The build API reported success within a minute, but different CDN edge nodes kept serving the previous version for several minutes after – with `age: 0` response headers that made it look like a fresh fetch of stale content. Verifying a deploy means comparing file hashes, not trusting a status field.

RISK	HANDLING	STATUS
Unsafe content reaches narration	System-prompt steering + LLM safety judge + 2 retries + bundled fallback story as last resort.	MITIGATED
Safety judge itself is wrong or unreachable	Fails open on a request error (story still plays) so a network hiccup can't strand the parent – but this means an unreachable judge is silently skipped, not silently blocked.	OPEN
API key exposure	Never committed to source; lives only in the visitor's own <code>localStorage</code> , sent only to OpenAI. Right model for one user, wrong model for many.	MITIGATED
Runaway API spend	None built in – no client-side rate limit or generation cap. Relies entirely on the spend limits the user sets on their own OpenAI account.	OPEN

RISK	HANDLING	STATUS
Single point of failure on OpenAI	All three calls hit one provider; no fallback provider, no server-side abstraction to swap one in.	OPEN
No audit trail for flagged content	Nothing is logged anywhere – good for privacy, bad for reviewing near-misses after the fact. Accepted tradeoff for a personal demo.	OPEN
BabyBedtimeStoryTeller · MVP architecture, as deployed github.com/adamhelwa/BabyBedtimeStoryTeller		